Translating UML Into Code

ICT 4315: Object Oriented Methods and Programming II

for

Master of Science

Information Technology

Kalika Browder

University of Denver College of Professional Studies

April 5, 2026

Faculty: Nathan Braun, MBA
Director: Cathie Wilson, M.S.
Dean: Michael J. McGuire, MLS

Problem Statement and Requirements

To begin, this assignment builds directly upon the object-oriented parking system that was previously developed in ICT-4305. This version of the system created already supports customer registration, vehicle registration, permit generation, and parking transaction tracking. The challenge presented in this iteration of the system was to modernize the communication layer between the parking system's command-line client and its server. The existing protocol communicated through a line-by-line text format, where the client would send a command name followed by key=value pairs, concluding with the word "end." While functional, this approach is actually brittle, difficult to extend, and incompatible with modern tooling like that of curl or web-based clients.

The goal was to replace this protocol with a JSON-based serialization layer. More specifically, two new classes were required to make this functional: ParkingRequest and ParkingResponse. These classes would encapsulate the data exchanged between client and server, and expose methods to convert themselves to and from JSON strings using the GSON library. The server would also be updated to receive a JSON-encoded ParkingRequest, process it, and return a JSON-encoded ParkingResponse. The client would be updated symmetrically. At a minimum, the CUSTOMER and CAR commands needed to be implemented through this new serialization mechanism, too.

Design Overview and Class Structure

The design centers around two new transfer objects and targeted modifications to the existing server and client classes. The broader class hierarchy from ICT-4305 remains intact, including ParkingOffice, PermitManager, TransactionManager, Customer, Car, ParkingPermit, ParkingTransaction, ParkingLot, ParkingCharge, Money, and Address.

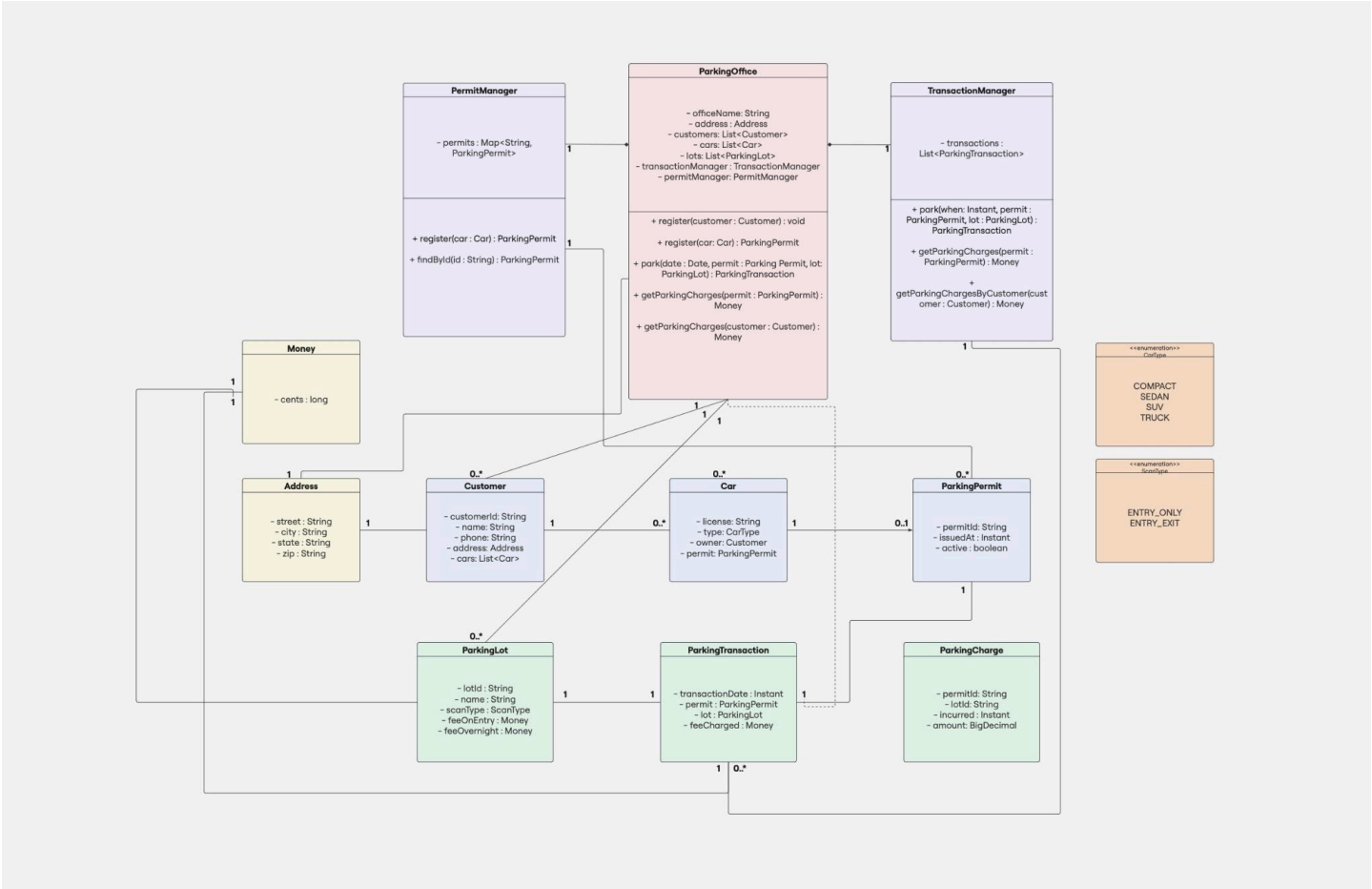

Figure 1: Final ICT-4305 Class Diagram

The new additions for this new version of the system and this assignment are described in the following:

ParkingRequest

This class represents an inbound request from the client. It holds two fields: a String commandName (like “CUSTOMER” or “CAR”) and a Properties object containing the parameters for the command. Two methods were added beyond the constructor and toString: to Json(), which serializes the object to a JSON string using GSON, and from Json(String), a static factory method that deserializes a JSON string back into a ParkingRequest. Because Properties is a Map subtype, GSON handles it well with minor type adapter configuration.

ParkingResponse

This class represents the server’s reply. It contains an integer statusCode (analogous to HTTP status codes, where 200 indicates success and 400 indicates a bad request) and a String message describing the result. Like ParkingRequest, it includes toJson() and fromJson(String) methods. This symmetry between the two classes made the protocol predictable and easy to test.

Server (Modified)

The server was updated to read a single JSON string from the client’s socket connection using a BufferedReader. It constructs a ParkingRequest via fromJson(), routes the command to the appropriate handler using the existing ParkingService command dispatch mechanism, and then wraps the result in a ParkingResponse, which is serialized to JSON and written back to the

client. Error handling was also added to return a ParkingResponse with statusCode 400 if the command was unrecognized or if required parameters were missing.

ServerClient (Modified)

The client was updated to construct a ParkingRequest from the command-line arguments, serialize it to JSON, and write it as a single line to the server's socket. It then reads the server's response as a JSON string and deserializes it into a ParkingResponse, printing the status code and the message to the console.

UML Diagram Description

The updated class diagram (see below + attached for closer look) reflects the additions to the system. ParkingRequest and ParkingResponse are shown as standalone classes with no inheritance relationships. Each exposes a constructor, a toString() override, a toJson() method returning String, and a static fromJson(String) method. ParkingRequest contains a commandName: String field and a parameters: Properties field. ParkingResponse contains statusCode: int and message: String. The Server class has a dependency on both ParkingRequest and ParkingResponse, as does the ServerClient class. ParkingService remains unchanged in its interface but is also called by the Server to perform the actual command logic.

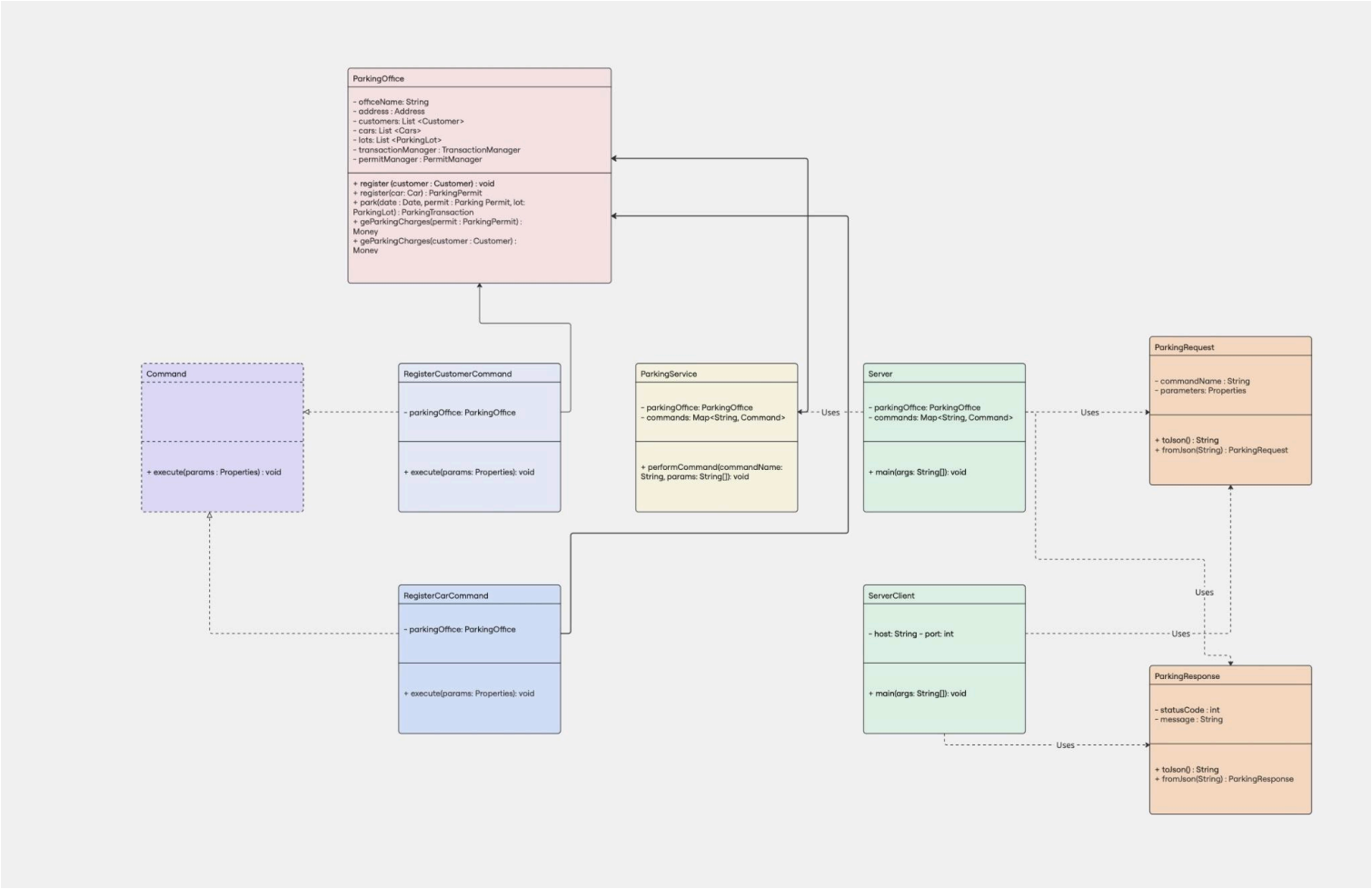

Figure 2: ICT-4315 Updated Class Diagram

Unit Tests

Unit tests were written for both ParkingRequest and ParkingResponse using JUnit 5. The test suite covers the following scenarios:

- ParkingRequestTest: Tests the constructor by verifying that commandName and parameters are correctly stored. Tests toString() for human-readable output. Tests toJson() to confirm the output is a valid JSON string containing the command name. Tests fromJson() using the output of toJson() as input, confirming round-trip fidelity (the deserialized object's commandName matches the original).
- ParkingResponseTest: Mirrors the above structure. Tests the constructor, toString(), toJson, and fromJson() for round-trip correctness. Also specifically asserts that a status code of 200 survives serialization and deserialization intact, and that the message field is preserved correctly.

Integration-level testing was performed manually by running the server and then invoking the client with CUSTOMER and CAR commands and confirming that the server printed and returned the correct responses.

Challenges, Lessons Learned, Conclusions, etc.

This assignment tested me thoroughly, especially because I am revisiting a system that I built two quarters ago, so it needed quite a bit of refreshing and modernizing many parts of the codebase to get it working. One of the first unexpected issues I came across was around the ParkingOffice class: the original design in ICT-4305 included methods like addLot(), register(Customer, String, CarType), getAllCharges(), and addCharge(), which were missing or inconsistent in my earlier updates. These inconsistencies cause compilation errors in both Main.java and RegisterCarCommand.java. Fixing them required restoring these methods while also keeping the internal state consistent and ensuring both cars and permits were properly registered.

A major challenge was GSON's handling of Properties. Since Properties extends Hashtable, the default serialization produced nested JSON structures that did not deserialize cleanly back into Properties. I resolved this by implementing minimal customer serialization/deserialization logic in ParkingRequest and ParkingResponse, allowing flat key-value JSON objects for command parameters. Once this worked, the JSON protocol between client and server became reliable.

Another challenge involved coordinating two layers of protocol: the command-line interface and the client-server communication. The command line still uses KEY=VALUE arguments, which are packaged into a ParkingRequest before being sent to the server.

Distinguishing these layers conceptually was critical, as it ensured that the command-line parsing and the server's JSON protocol remained compatible and testable.

Socket stream handling presented further hardships for me. Using BufferedReader and PrintWriter required careful attention to flushing behavior. Initially, the server would sometimes block waiting for input that the client had already written. Enabling autoFlush and sending exactly one JSON line per request resolved this, which reinforced the importance of managing stream buffers in network communication.

From a design perspective, this assignment highlighted the value of transfer objects. Encapsulating client requests and server responses in ParkingRequest and ParkingResponse improved readability, maintainability, and testability. Integrating this layer on top of a stable domain model allowed me to focus on communication logic without breaking the core parking functionality.

Finally, Main.java served as a practical integration test, demonstrating customer registration, car registration, parking entries/exits, and charge calculations. Fixing the ParkingOffice inconsistencies ensured that all interactions, including office.addLot(), office.register(Customer, String, CarType), and office.getAllCharges(), worked as expected. This iteration reinforced the overarching lesson: a clean, well-tested domain model makes adding new layers and features far easier, reducing debugging overhead and supporting rapid integration of client-server communication.

Also for reference:

Java version: 17 (documented in pom.xml). JUnit version: 5.9.2. GSON version: 2.10.1. Main entry points:

Server.java (server process) and ServerClient.java (CLI client). Example commands: java -cp target/classes
edu.du.ict4315.parking.server.Server | java -cp target/classes
edu.du.ict4315.parking.clients.ServerClient
CUSTOMER firstname=Kalika | java -cp target/classes
edu.du.ict4315.parking.clients.ServerClient CAR
license=KAL4CO customer=CUST1